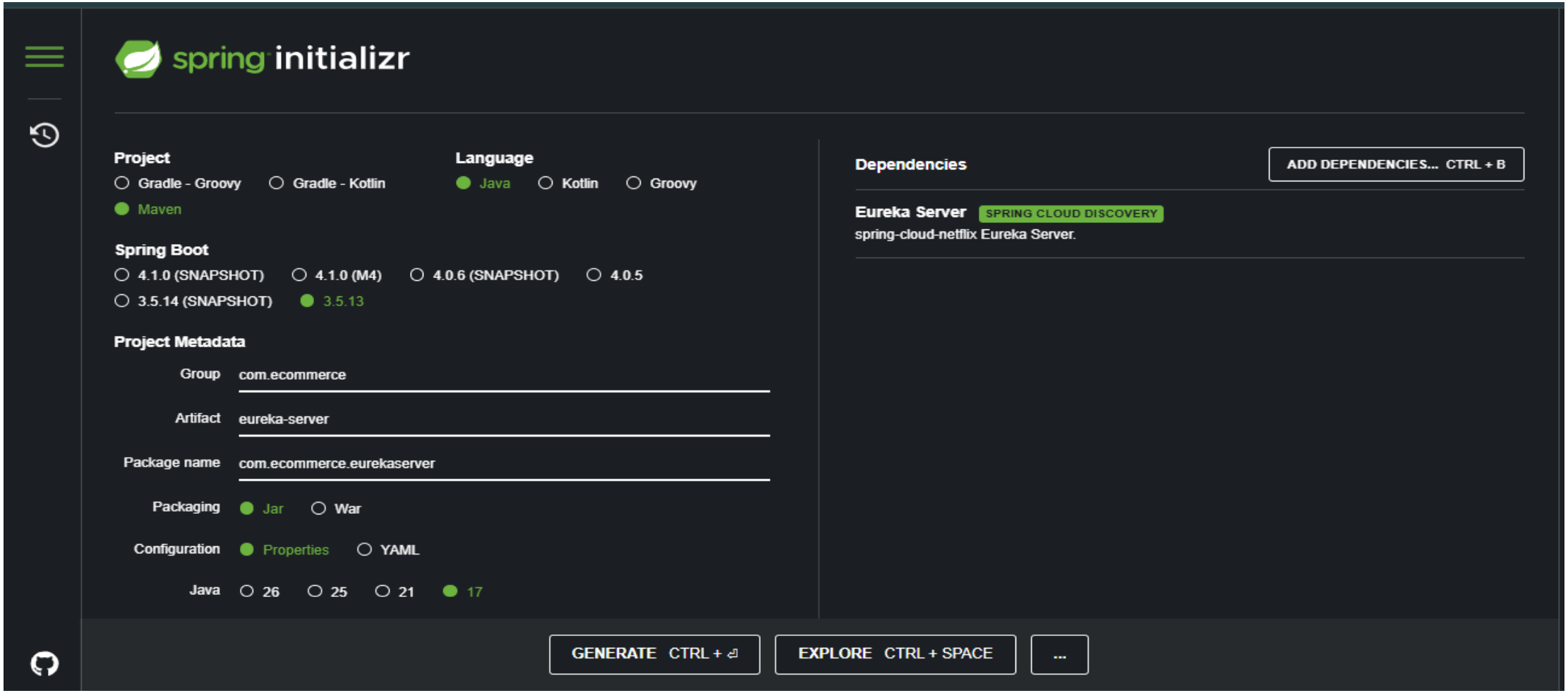
TP°4 : Architecture Orientée Services (SOA)

Encadré par: Pr. BE ELBAGHAZAOUI

Réalisé par: Nabila Abouilaaz

Étape 1 : Créer le serveur Eureka
1-Création du projet



2. Configuration
Le fichier application.properties :

application.properties × EurekaServerApplication.java

```
1  server.port=8761
2  spring.application.name=eureka-server
3  eureka.client.register-with-eureka=false
4  eureka.client.fetch-registry=false
5  eureka.client.service-url.defaultZone=http://localhost:8761/eureka/spring.application.name=eureka-server
6
```

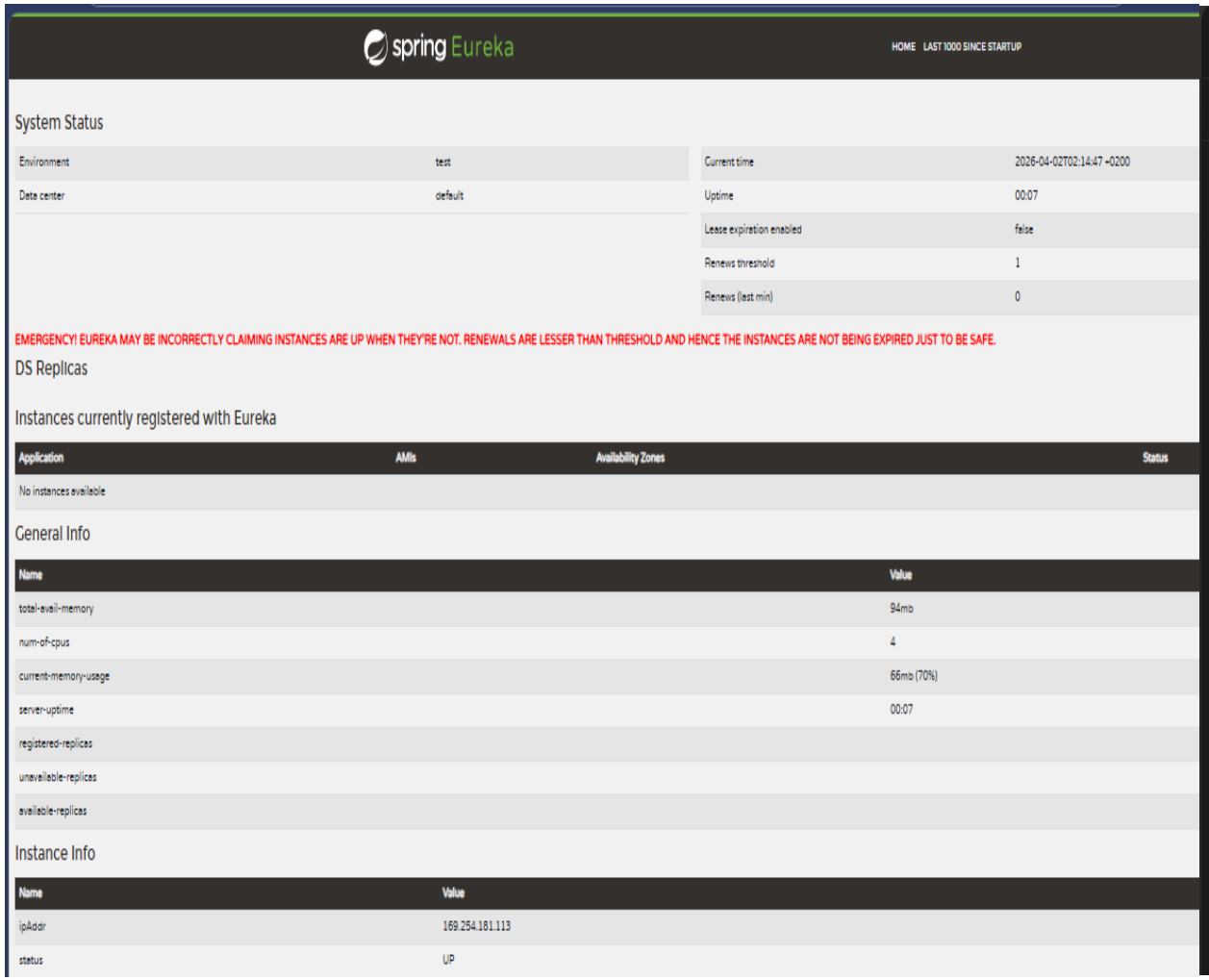
3. Classe principale

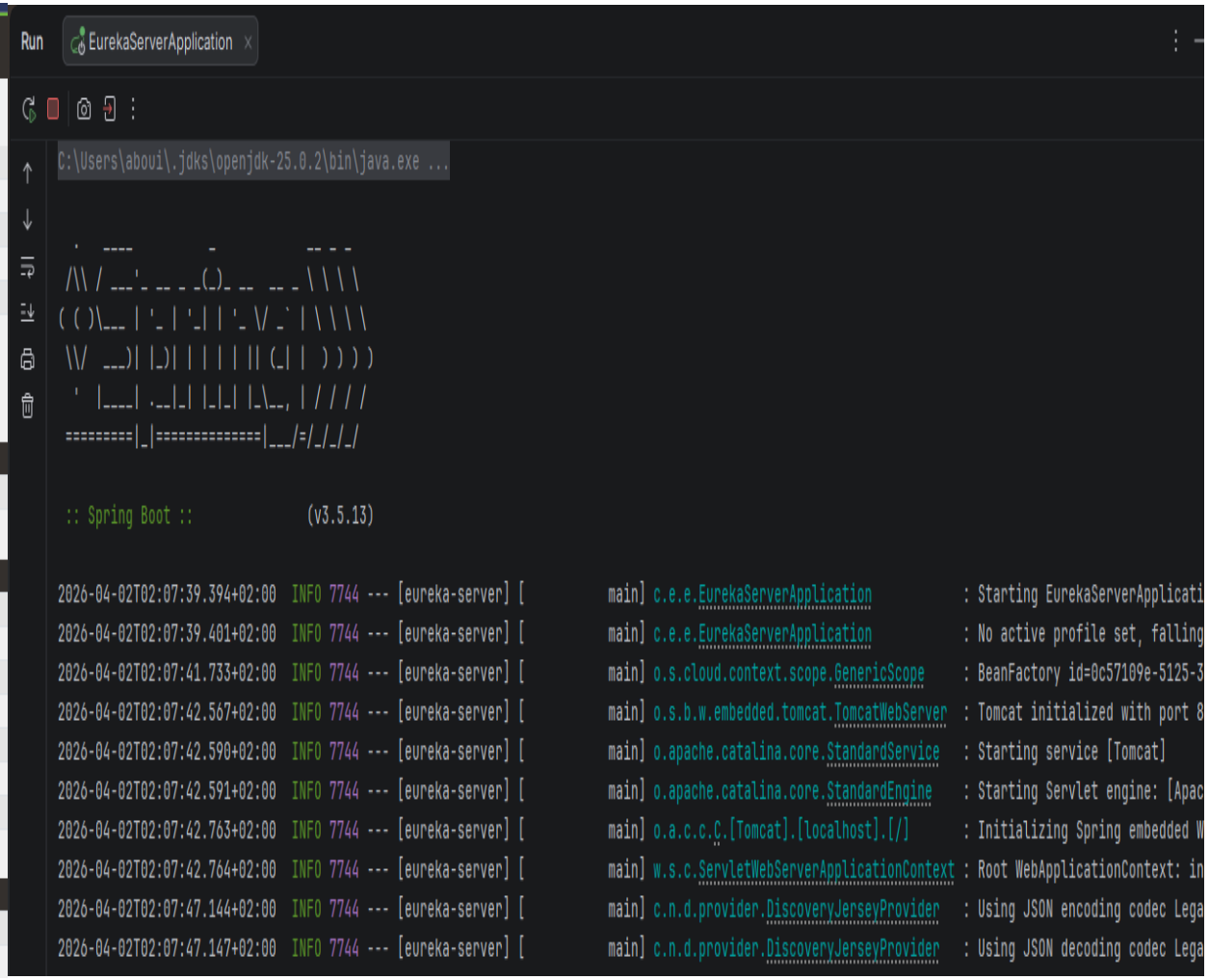
Ajout de l'annotation @EnableEurekaServer dans la classe principale :

application.properties | EurekaServerApplication.java

```java
package com.ecommerce.eurekaserver;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

4 . Démarrage


spring Eureka
HOME LAST 1000 SINCE STARTUP
System Status
Environment test
Data center default
Current time 2026-04-02T02:14:47 +0200
Uptime 00:07
Lease expiration enabled false
Renews threshold 1
Renews (last min) 0
EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.
DS Replicas
Instances currently registered with Eureka
Application AMIs Availability Zones Status
No instances available
General Info
Name Value
total-avail-memory 94mb
num-of-cpus 4
current-memory-usage 66mb (70%)
server-uptime 00:07
registered-replicas
unavailable-replicas
available-replicas
Instance Info
Name Value
ipAddr 169.254.181.113
status UP


Run EurekaServerApplication
C:\Users\aboui\.jdks\openjdk-25.0.2\bin\java.exe ...
:: Spring Boot :: (v3.5.13)
2026-04-02T02:07:39.394+02:00 INFO 7744 --- [eureka-server] [main] c.e.e.EurekaServerApplication : Starting EurekaServerApplicati
2026-04-02T02:07:39.401+02:00 INFO 7744 --- [eureka-server] [main] c.e.e.EurekaServerApplication : No active profile set, falling
2026-04-02T02:07:41.733+02:00 INFO 7744 --- [eureka-server] [main] o.s.cloud.context.scope.GenericScope : BeanFactory id=0c57109e-5125-3
2026-04-02T02:07:42.567+02:00 INFO 7744 --- [eureka-server] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8
2026-04-02T02:07:42.590+02:00 INFO 7744 --- [eureka-server] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-04-02T02:07:42.591+02:00 INFO 7744 --- [eureka-server] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apac
2026-04-02T02:07:42.763+02:00 INFO 7744 --- [eureka-server] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded W
2026-04-02T02:07:42.764+02:00 INFO 7744 --- [eureka-server] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: in
2026-04-02T02:07:47.144+02:00 INFO 7744 --- [eureka-server] [main] c.n.d.provider.DiscoveryJerseyProvider : Using JSON encoding codec Lega
2026-04-02T02:07:47.147+02:00 INFO 7744 --- [eureka-server] [main] c.n.d.provider.DiscoveryJerseyProvider : Using JSON decoding codec Lega

Étape 2 : Créer le service Product
1. Création du projet

spring initializr

Project
- Gradle - Groovy
- Gradle - Kotlin
- Maven

Language
- Java
- Kotlin
- Groovy

Spring Boot
- 4.1.0 (SNAPSHOT)
- 4.1.0 (M4)
- 4.0.6 (SNAPSHOT)
- 4.0.5
- 3.5.14 (SNAPSHOT)
- 3.5.13

Project Metadata

Group	com.ecommerce
Artifact	product-service
Package name	com.ecommerce.productservice
Packaging	Jar / War
Configuration	Properties / YAML
Java	26 / 25 / 21 / 17

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Eureka Discovery Client SPRING CLOUD DISCOVERY
A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Lombok DEVELOPER TOOLS
Java annotation library which helps to reduce boilerplate code.

GENERATE CTRL + ⏎ | EXPLORE CTRL + SPACE | ...

2. Configuration
Fichier application.properties :

product-service\...\application.properties

```
server.port=8081
spring.application.name=product-service eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
# H2 Database
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update spring.h2.console.enabled=true
```

3. Modèle de données
Création de la classe Product.java :

application.properties | ProductServiceApplication.java | Product.java

```java
package com.ecommerce.productservice.model;
import jakarta.persistence.*;
import lombok.*;
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

4. Repository
Création ProductRepository.java :

application.properties | ProductServiceApplication.java | Product.java | ProductRepository.java

```java
package com.ecommerce.productservice.repository;
3 / 6
import com.ecommerce.productservice.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> { }
```

5. Service

Création ProductService.java :

pom.xml (product-service) | ProductService.java | ProductController.java | Product.java | Pr

```java
package com.ecommerce.productservice.service;
import com.ecommerce.productservice.model.Product;
import com.ecommerce.productservice.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List; import java.util.Optional;
@Service
@RequiredArgsConstructor
public class ProductService {
    private final ProductRepository productRepository;
    public List<Product> getAllProducts() { return productRepository.findAll(); }  1 usage
    public Optional<Product> getProductById(Long id) { return productRepository.findById(id); }  1 u
    public Product createProduct(Product product) { return productRepository.save(product); }  1 usa
    public void deleteProduct(Long id) { productRepository.deleteById(id); }  1 usage
}
```

6. Contrôleur REST

Création ProductController.java :

application.properties | ProductServiceApplication.java | ProductController.java | Product.java | ProductRepository.java

```java
package com.ecommerce.productservice.controller;
import com.ecommerce.productservice.model.Product;
import com.ecommerce.productservice.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;
    @GetMapping public List < Product > getAllProducts() {
        return productService.getAllProducts();
    }
    @GetMapping("/{id}") public ResponseEntity < Product > getProductById(@PathVariable Long id) {
        return productService.getProductById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());
    }
    @PostMapping public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }
    @DeleteMapping("/{id}") public ResponseEntity < Void > deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

7. Activation Eureka Client
Ajout @EnableDiscoveryClient dans la classe principale :

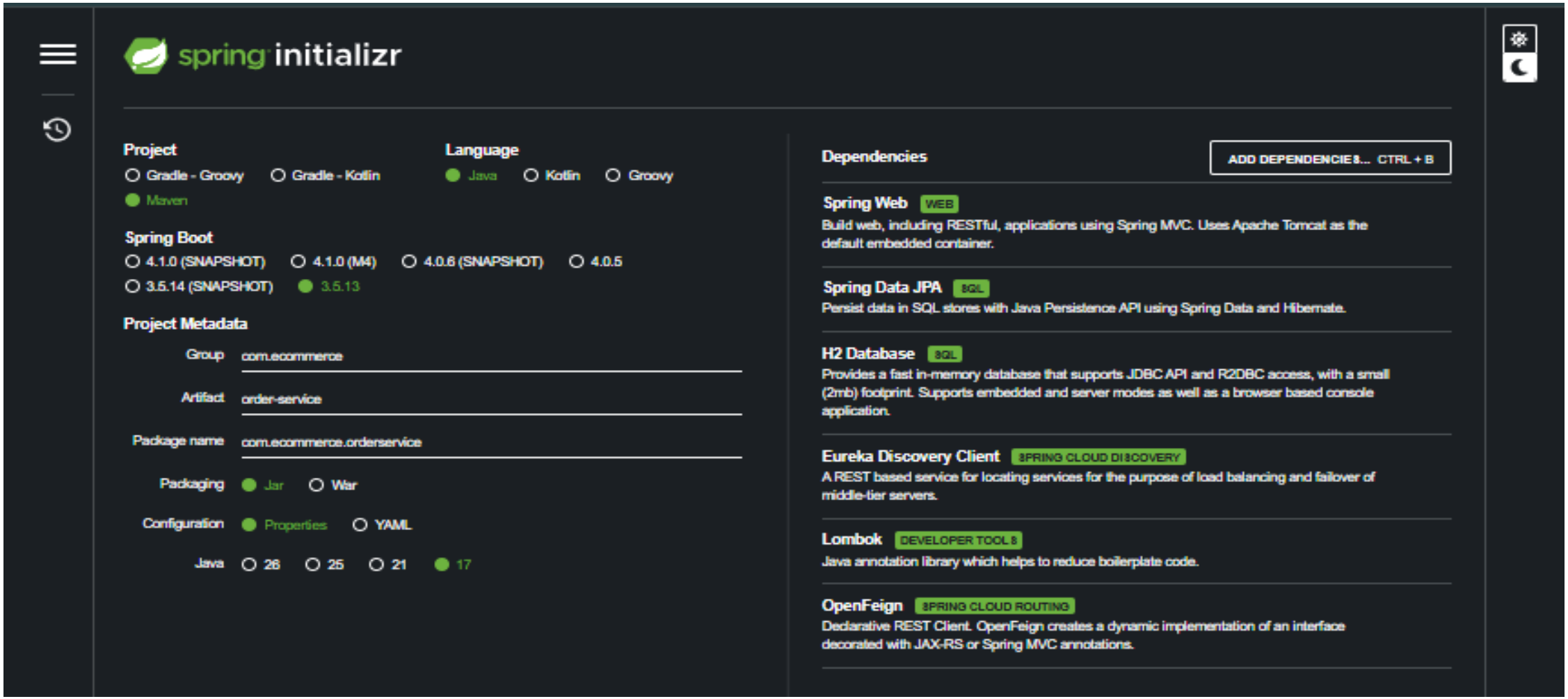
ProductServiceApplication.java | mvnw.cmd | pom.xml (product-service) | ProductService.java | ProductCon

```java
package com.ecommerce.productservice;

import ...

@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) { SpringApplication.run(ProductServiceApplication.class, args); }
}
```

Étape 3 : Créer le service Order
1. Création du projet



2. Configuration

pplication.java | order-service\...\application.properties | mvnw.cmd | pom.xml (product-service) | ProductService.java

```
server.port=8082
spring.application.name=order-service eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

3. Modèle de données

pplication.java | order-service\...\application.properties | Order.java | mvnw.cmd

```java
package com.ecommerce.orderservice.Model;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status; // PENDING, CONFIRMED, SHIPPED, DELIVERED
}
```

4. Client Feign pour Product Service

Créer ProductClient.java pour communiquer avec Product Service :

pplication.java | ProductDTO.java | ProductClient.java | mvnw.cmd | pom.xml (product-service

```java
package com.ecommerce.orderservice.client;
import com.ecommerce.orderservice.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
@FeignClient(name = "product-service")  no usages
public interface ProductClient {
    @GetMapping("/api/products/{id}") ProductDTO getProductById(@PathVariable Long id);
}
```

5. DTO pour Product

Application.java | ProductDTO.java | mvnw.cmd

```java
package com.ecommerce.orderservice.dto;
import lombok.*;
@Data  no usages
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

6. Repository et Service
OrderRepository.java :

```
pplication.java    ProductDTO.java    ProductClient.java    OrderRepository.java

1  package com.ecommerce.orderservice.Repository;
2  import com.ecommerce.orderservice.Model.Order;
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import org.springframework.stereotype.Repository;
5  @Repository  no usages
6  public interface OrderRepository extends JpaRepository < Order, Long > {}
```

OrderService.java :

pplication.java | ProductDTO.java | ProductClient.java | OrderRepository.java | OrderService.java

```java
package com.ecommerce.orderservice.service;
import com.ecommerce.orderservice.client.ProductClient;
import com.ecommerce.orderservice.dto.ProductDTO;
import com.ecommerce.orderservice.Model.Order;
import com.ecommerce.orderservice.Repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductClient productClient;
    public List < Order > getAllOrders() {  no usages
        return orderRepository.findAll();
    }
    public Order createOrder(Long productId, Integer quantity) {  no usages
        ProductDTO product = productClient.getProductById(productId);
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
```

7. Contrôleur

roductDTO.java × | ProductClient.java | OrderRepository.java | OrderService.java | OrderController.java

```java

import com.ecommerce.orderservice.Model.Order;
import com.ecommerce.orderservice.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @GetMapping
    public List<Order> getAllOrders() {
        return orderService.getAllOrders();
    }

    @PostMapping
    public Order createOrder(@RequestParam Long productId,
                             @RequestParam Integer quantity) {
        return orderService.createOrder(productId, quantity);
    }
}
```

8. Activation des clients Feign

TO.java | ProductClient.java | OrderRepository.java | OrderService.java | OrderServiceApplication.java

```java
package com.ecommerce.orderservice;

import ...

@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Étape 4 : Tester l'application

1. Démarrage des services

Démarrage des services dans l'ordre suivant :

9. Eureka Server (port 8761)
10. Product Service (port 8081)
11. Order Service (port 8082)

Vérifier l'enregistrement :

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-04-03T16:41:18 +0200
Data center	default	Uptime	00:02
		Lease expiration enabled	false
		Renews threshold	5
		Renews (last min)	3

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - localhost:product-service:8081

General Info

Name	Value
total-avail-memory	100mb
num-of-cpus	4
current-memory-usage	43mb (43%)
server-uptime	00:02
registered-replicas	
unavailable-replicas	

localhost:8081/actuator/info

Pretty-print

{"app":{"name":"product-service","version":"0.0.1-SNAPSHOT","description":"Product microservice"}}

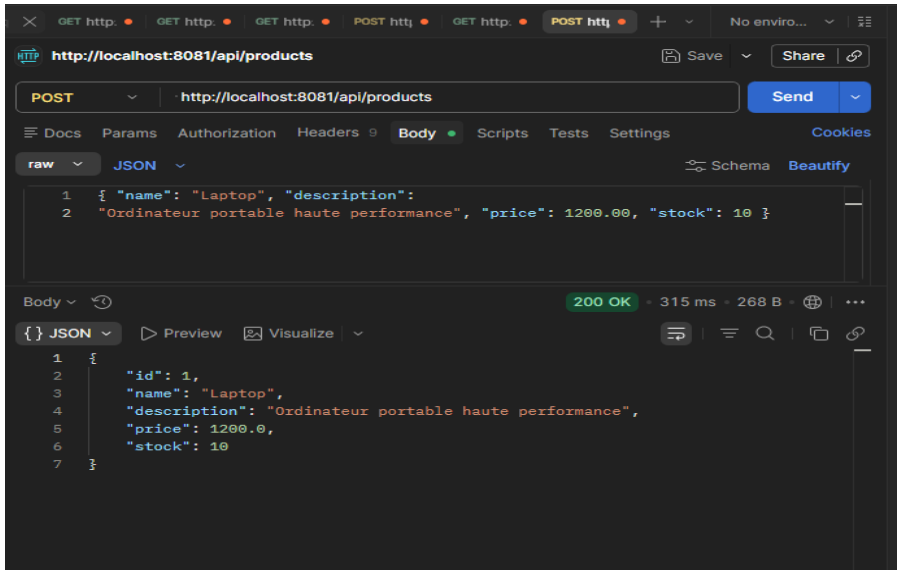
localhost:8082/actuator/info

Pretty-print

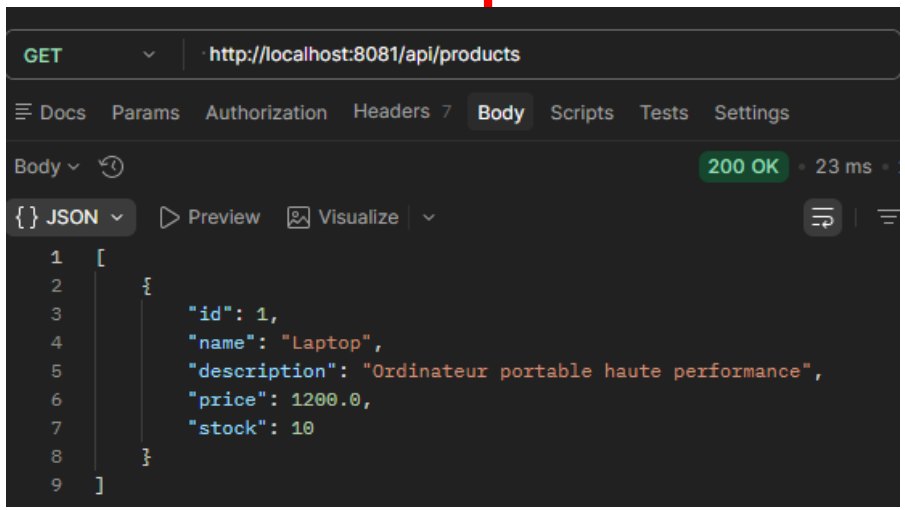
{"app":{"description":"Order microservice","version":"0.0.1-SNAPSHOT","name":"order-service"}}

2. Tests avec Postman

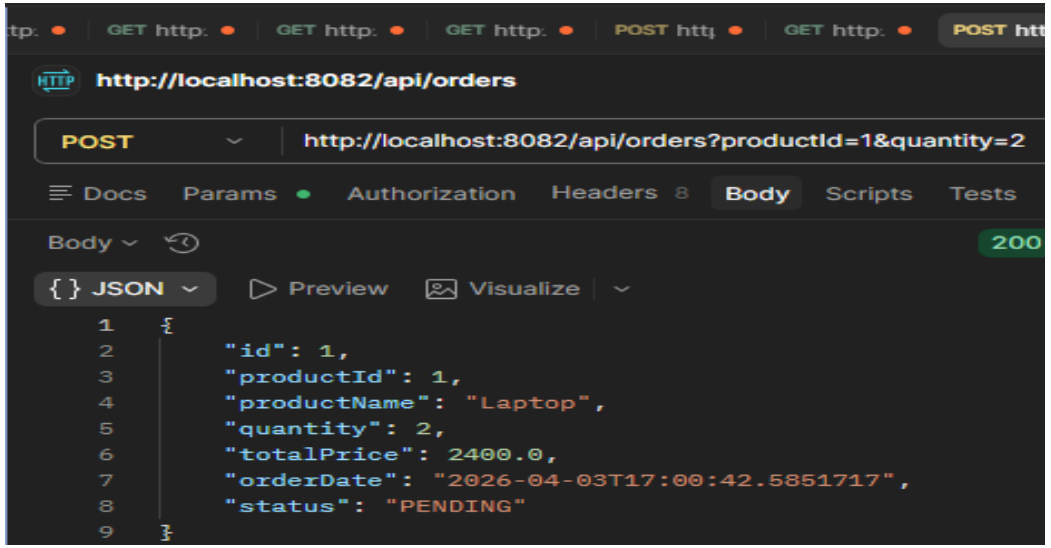
Test 1 : Créer un produit



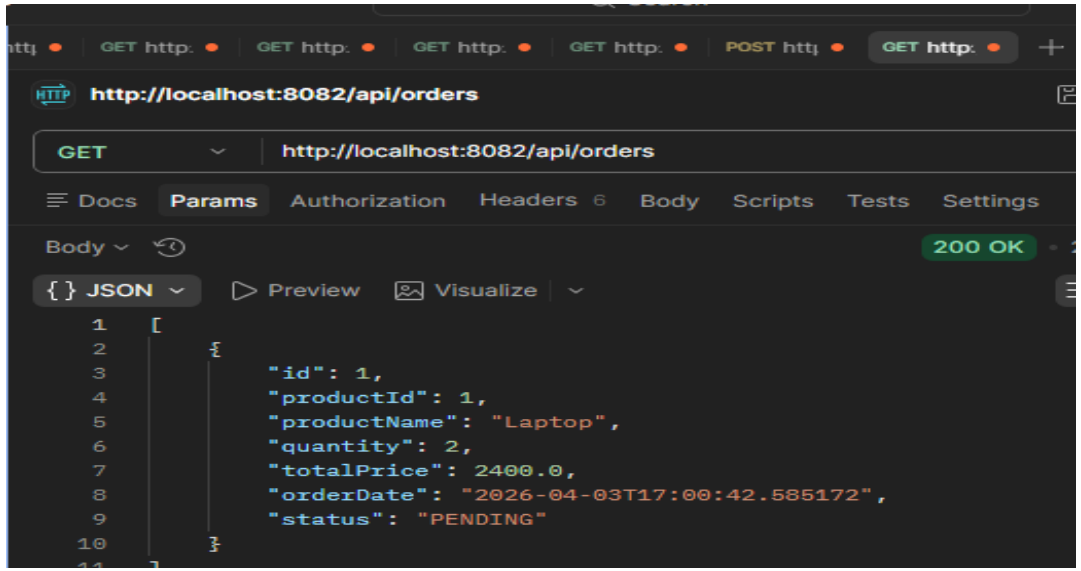
Test 2 : Lister les produits



Test 3 : Créer une commande



Test 4 : Lister les commandes



7. Exercices pratiques

Exercice 1 : Ajout d'une fonctionnalité

Ajout d'une méthode dans Product Service pour mettre à jour le stock après une commande

Dans le fichier ProductService.java .

```java
    // Exercice 1 : Mise à jour du stock après une commande
    public Product updateStock(Long id, Integer quantityOrdered) {  1 usage
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Produit introuvable avec l'id : " + id));

        if (product.getStock() < quantityOrdered) {
            throw new RuntimeException("Stock insuffisant. Stock disponible : " + product.getStock()
                    + ", quantité demandée : " + quantityOrdered);
        }

        product.setStock(product.getStock() - quantityOrdered);
        return productRepository.save(product);
    }
}
```

Dans le fichier ProductController.java

```java
    // Exercice 1 : Endpoint PUT /api/products/{id}/stock
    @PutMapping("/{id}/stock")
    public ResponseEntity<?> updateStock(@PathVariable Long id,
                                         @RequestParam Integer quantity) {
        try {
            Product updated = productService.updateStock(id, quantity);
            return ResponseEntity.ok(updated);
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}
```

3. Modification de Order Service pour appeler cet endpoint après création d'une commande

Dans le fichier ProductClient.java

```java
    // Exercice 1 : Appel du endpoint PUT /api/products/{id}/stock
    @PutMapping("/api/products/{id}/stock")  1 usage
    void updateStock(@PathVariable Long id, @RequestParam Integer quantity);
}
```

Dans le fichier OrderService.java:

```java
        // Exercice 1 : Vérifier le stock disponible avant de créer la commande
        if (product.getStock() < quantity) {
            throw new InsufficientStockException(productId, quantity);
        }
        // Créer la commande
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        Order savedOrder = orderRepository.save(order);
        // Exercice 1 : Mettre à jour le stock dans Product Service
        try {
            productClient.updateStock(productId, quantity);
        } catch (FeignException e) {
            // Si la mise à jour du stock échoue, on annule la commande
            orderRepository.delete(savedOrder);
            throw new ProductServiceUnavailableException();
        }
        return savedOrder;
    }}
```

Exercice 2 : Créer une API Gateway

Création un service API Gateway avec Spring Cloud Gateway.

OrderService.java | ApiGatewayApplication.java | application.yml | OrderServiceApplication.java

```java
package com.ecommerce.apigateway;

import ...

@SpringBootApplication
@EnableDiscoveryClient
public class ApiGatewayApplication {
    public static void main(String[] args) { SpringApplication.run(ApiGatewayApplication.class, args); }
}
```

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test
Data center	default

Current time	2026-04-03T18:06:24 +0200
Uptime	00:17
Lease expiration enabled	false
Renews threshold	6
Renews (last min)	3

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - localhost:api-gateway:8080
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - localhost:product-service:8081

localhost:8080/actuator/info

Pretty-print


```
{
  "app": {
    "name": "api-gateway",
    "description": "Point d'entrée unique - API Gateway",
    "version": "0.0.1-SNAPSHOT"
  }
}
```


Exercice 3 : Gestion des erreurs

```
// Exercice 3 : Gérer le cas où le service Product est indisponible
try {
    product = productClient.getProductById(productId);
} catch (FeignException.NotFound e) {
    // Exercice 3 : Gérer le cas où le produit n'existe pas
    throw new ProductNotFoundException(productId);
} catch (FeignException e) {
    throw new ProductServiceUnavailableException();
}
```

POST http://localhost:8080/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Body 200 OK 193 ms 220 B

{} JSON Preview Visualize


```
{
    "id": 1,
    "name": "Laptop",
    "description": "Ordinateur portable haute performance",
    "price": 1200.0,
    "stock": 10
```


http://localhost:8080/products

GET http://localhost:8080/products

Docs Params Authorization Headers 9 Body Scripts Tests Settings

Body 200 OK 404

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Laptop",
        "description": "Ordinateur portable haute performance",
        "price": 1200.0,
        "stock": 10
    }
]
```